

// Jivops

Guía completa — 30 lecciones

Arquitectura · CRI y Kubernetes · Imágenes y snapshotters · Runtimes
y seguridad · Operación a escala · Producción avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es containerd y su rol como runtime de bajo nivel
- Lección 02 — Arquitectura de containerd — núcleo y plugins
- Lección 03 — containerd vs Docker vs CRI-O — el panorama de runtimes
- Lección 04 — Instalación y configuración de containerd
- Lección 05 — Namespaces internos de containerd (distinto de namespaces de Linux)

Módulo 2: CRI e integración con Kubernetes

- Lección 06 — Container Runtime Interface (CRI) — el contrato con Kubernetes
- Lección 07 — containerd como runtime de kubelet
- Lección 08 — crictl — CLI para depurar containerd vía CRI
- Lección 09 — Configuración de containerd para Kubernetes (config.toml)
- Lección 10 — Sandbox/pause container — cómo containerd gestiona Pods

Módulo 3: Imágenes y snapshotters

- Lección 11 — ctr — CLI de bajo nivel de containerd
- Lección 12 — Pull y gestión de imágenes con containerd
- Lección 13 — Snapshotters — overlayfs y alternativas
- Lección 14 — Content store — almacenamiento de contenido de imágenes
- Lección 15 — Garbage collection de imágenes y contenido no usado

Módulo 4: Runtimes y seguridad

- Lección 16 — runc y la interfaz OCI Runtime
- Lección 17 — Runtimes alternativos — Kata Containers, gVisor (runsc)
- Lección 18 — RuntimeClass en Kubernetes — múltiples runtimes en el mismo cluster
- Lección 19 — Configuración de runtime seguro para cargas no confiables
- Lección 20 — Seccomp, AppArmor y containerd

Módulo 5: Operación a escala

- Lección 21 — Plugins de containerd — extensibilidad
- Lección 22 — Configuración de registries y mirrors
- Lección 23 — Métricas de containerd (Prometheus)
- Lección 24 — Troubleshooting con ctr y crictl
- Lección 25 — Logs y debugging de containerd

Módulo 6: Producción avanzada

Lección 26 — Actualización de containerd sin downtime de Pods

Lección 27 — Rendimiento — tuning de containerd a gran escala

Lección 28 — Migración de dockershim a containerd

Lección 29 — containerd en entornos air-gapped

Lección 30 — Proyecto final: runtime de contenedores completo con containerd listo para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es containerd y su rol como runtime de bajo nivel

containerd está pensado para ser controlado vía API por otro sistema como Kubernetes, no para uso interactivo humano diario como Docker.

Lección 02 — Arquitectura de containerd — núcleo y plugins

El sistema de plugins permite intercambiar el snapshotter o runtime más adecuado por entorno sin modificar el núcleo central del proyecto.

Lección 03 — containerd vs Docker vs CRI-O — el panorama de runtimes

containerd es también el runtime que usa Docker internamente, dándole un alcance más general que CRI-O, enfocado específicamente en Kubernetes.

Lección 04 — Instalación y configuración de containerd

Generar la configuración con 'containerd config default' evita errores de sintaxis frente a escribir el config.toml completo manualmente desde cero.

```
containerd config default
```

Lección 05 — Namespaces internos de containerd (distinto de namespaces de Linux)

El namespace 'k8s.io' aísla lógicamente lo que gestiona Kubernetes de lo que gestionaría Docker en el mismo containerd, evitando conflictos.

```
ctr --namespace k8s.io
```

Módulo 2: CRI e integración con Kubernetes

Lección 06 — Container Runtime Interface (CRI) — el contrato con Kubernetes

El estándar CRI permite a kubelet hablar con cualquier runtime compatible usando la misma lógica genérica, sin código específico por cada uno.

Lección 07 — containerd como runtime de kubelet

El plugin CRI integrado en containerd simplifica la instalación, reduciendo piezas separadas que coordinar frente a un componente completamente aparte.

Lección 08 — crictl — CLI para depurar containerd vía CRI

crictl interactúa directamente a nivel del nodo, dando visibilidad de detalles que kubectl, más abstracto, podría no mostrar.

```
crictl ps
```

Lección 09 — Configuración de containerd para Kubernetes (config.toml)

Usar el sandbox_image compatible con la versión de Kubernetes instalada evita errores al crear Pods por discrepancia de versión.

```
sandbox_image = "registry.k8s.io/pause:3.9"
```

Lección 10 — Sandbox/pause container — cómo containerd gestiona Pods

El namespace de red está anclado al pause container; la IP del Pod persiste aunque un contenedor de aplicación se reinicie internamente.

Módulo 3: Imágenes y snapshotters

Lección 11 — ctr — CLI de bajo nivel de containerd

Manipular con ctr recursos que Kubernetes también gestiona puede crear inconsistencias, ya que Kubernetes no se entera de ese cambio manual.

```
ctr images ls
```

Lección 12 — Pull y gestión de imágenes con containerd

Las capas se identifican por hash; si dos imágenes comparten la misma capa base, containerd la reutiliza sin descargarla por duplicado.

```
ctr images pull docker.io/library/nginx:latest
```

Lección 13 — Snapshotters — overlayfs y alternativas

overlayfs comparte capas de solo lectura entre contenedores, escribiendo solo los cambios en una capa fina propia de cada uno.

Lección 14 — Content store — almacenamiento de contenido de imágenes

Usar el hash del contenido como identificador detecta automáticamente cualquier corrupción, ya que el hash cambiaría si el contenido se alterara.

Lección 15 — Garbage collection de imágenes y contenido no usado

Sin limpieza periódica, cada nueva versión de imagen deja capas antiguas sin referencia que se acumulan indefinidamente en disco.

Módulo 4: Runtimes y seguridad

Lección 16 — runc y la interfaz OCI Runtime

Al implementar el estándar OCI Runtime, un runtime alternativo puede reemplazar a runc sin que containerd necesite cambios para invocarlo.

Lección 17 — Runtimes alternativos — Kata Containers, gVisor (runsc)

Un kernel dedicado por contenedor en Kata mitiga que una carga no confiable comprometa el kernel compartido y afecte a otros contenedores.

Lección 18 — RuntimeClass en Kubernetes — múltiples runtimes en el mismo cluster

Asignar un runtime más aislado solo a cargas menos confiables evita pagar ese overhead adicional en todo el cluster innecesariamente.

```
runtimeClassName: gvisor
```

Lección 19 — Configuración de runtime seguro para cargas no confiables

Una plataforma que ejecuta código arbitrario de clientes distintos y no relacionados es un caso ideal para aplicar un runtime como gVisor.

Lección 20 — Seccomp, AppArmor y containerd

Bloquear syscalls innecesarias reduce las opciones disponibles para un atacante que hubiera comprometido un proceso dentro del contenedor.

Módulo 5: Operación a escala

Lección 21 — Plugins de containerd — extensibilidad

Un snapshotter personalizado permite adaptar containerd a infraestructura de almacenamiento muy específica más allá de las opciones estándar.

Lección 22 — Configuración de registries y mirrors

Un mirror local evita que decenas de nodos descarguen independientemente la misma imagen popular, cacheándola una vez para todos.

```
endpoint = ["http://mirror-local:5000"]
```

Lección 23 — Métricas de containerd (Prometheus)

Monitorear métricas del runtime detecta problemas a nivel de infraestructura antes de que se hagan visibles como fallos en las aplicaciones.

Lección 24 — Troubleshooting con ctr y crictl

crictl refleja la misma vista que kubelet usa internamente, más relevante para diagnosticar problemas específicos de gestión de Pods.

```
crictl inspectp mi-pod-id
```

Lección 25 — Logs y debugging de containerd

Un problema a nivel del propio servicio (como fallo de conexión a un registry) se registra en sus logs, no en los de un contenedor específico.

```
journalctl -u containerd
```


Módulo 6: Producción avanzada

Lección 26 — Actualización de containerd sin downtime de Pods

El desacoplamiento entre containerd y los procesos vía runc evita que reiniciar el servicio para actualizar interrumpa los contenedores activos.

Lección 27 — Rendimiento — tuning de containerd a gran escala

Sin límite de concurrencia, demasiados pulls simultáneos podrían saturar el ancho de banda o I/O de disco compartido del nodo.

Lección 28 — Migración de dockershim a containerd

Eliminar la capa de traducción intermedia simplifica la arquitectura, reduciendo puntos potenciales de fallo entre kubelet y la ejecución real.

Lección 29 — containerd en entornos air-gapped

Un registry local sincronizado periódicamente da una fuente centralizada de imágenes, sin depender de acceso directo constante a internet.

Lección 30 — Proyecto final: runtime de contenedores completo con containerd listo para producción

Se integra todo lo anterior: configuración CRI correcta, snapshotter apropiado, RuntimeClass para cargas sensibles, registry mirrors, monitoreo con Prometheus y garbage collection configurado — el nivel esperado de un runtime containerd listo para producción real.